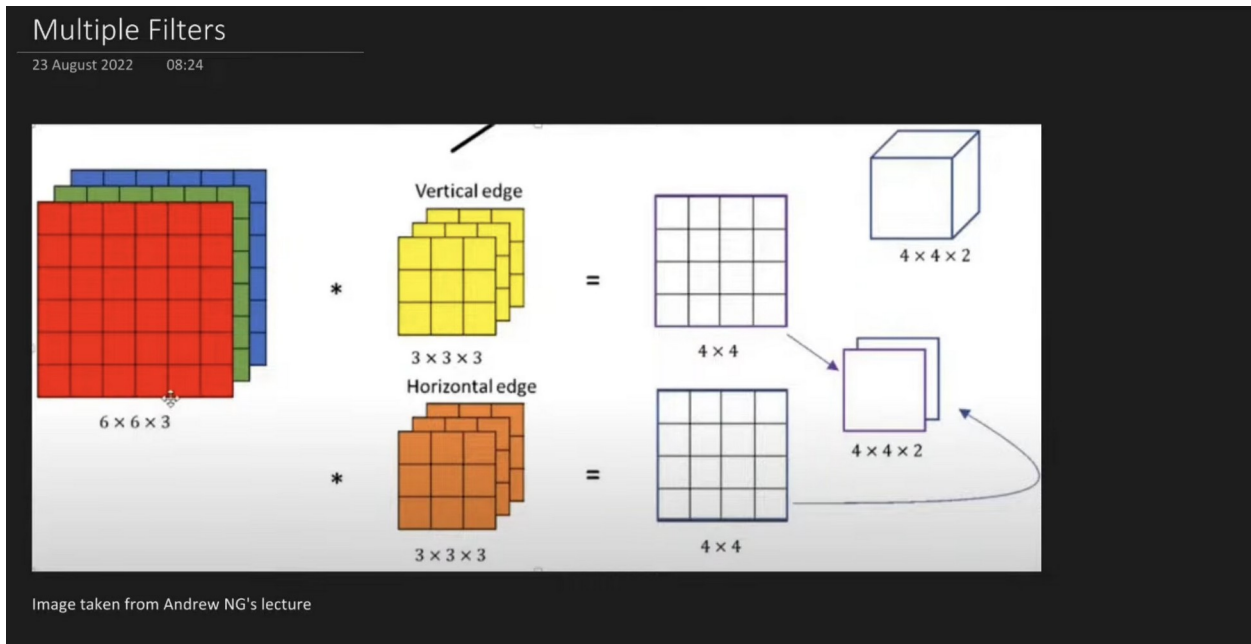


Topic 1: Introduction to CNNs & Image Representation



1. Introduction

What is a CNN? CNN stands for **Convolutional Neural Network** – a specialized type of neural network designed to work exceptionally well with **image data**.

Why is it important?

- CNNs are the backbone of modern computer vision systems
- They power facial recognition, self-driving cars, medical image analysis, and more
- Their architecture is inspired by the human visual cortex

Real-life use:

- Face detection in smartphones
 - Object recognition in autonomous vehicles
 - Medical diagnosis from X-rays and MRIs
 - Image classification (e.g., identifying cats vs. dogs)
-

2. Detailed Explanation

How CNNs Differ from Standard ANNs

Traditional Artificial Neural Networks (ANNs) treat every pixel as an independent input feature. This becomes impractical for images because:

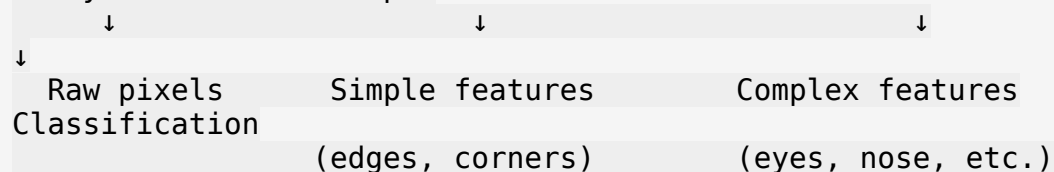
- A small 100×100 image has 10,000 pixels – too many connections
- ANNs lose spatial relationships between pixels

CNNs solve this by using a special architecture with different types of layers:

1. **Convolutional layers** – extract features from images
2. **Pooling layers** – reduce dimensions
3. **Fully connected layers** (dense layers) – perform classification

High-Level Overview of CNN Working

Input Image → [Convolution + Pooling] → [Convolution + Pooling] → Fully Connected → Output



The key insight:

- **Early layers** detect primitive features (edges, corners, color changes)
- **Deeper layers** combine these to detect more complex features (shapes, objects)
- **Final layers** make decisions (classification)

Analogy – Face Detection:

1. Layer 1 detects edges and lines
 2. Layer 2 combines edges to form shapes (eyes, nose, mouth)
 3. Layer 3 combines these to recognize a complete face
 4. Output layer decides if it's the face we're looking for
-

3. Key Points

- CNN is a neural network specialized for image processing
 - Architecture differs from ANN with specialized layers
 - Works by progressively extracting features from simple to complex
 - The **convolution operation** is the most critical component
 - CNNs learn feature filters automatically during training
-

4. Revision Notes

Concept	Summary
CNN	Neural network for images
Key layers	Convolution, Pooling, Fully Connected
Progression	Simple features → Complex features → Classification
Inspiration	Human visual cortex
Main advantage	Preserves spatial relationships in images

That covers the introduction to CNNs and image representation.

Topic 2: Image Representation in Computers

1. Introduction

What is this topic about? Before understanding how convolution works, we need to understand how computers actually "see" and store images. Images aren't pictures to a computer – they are **numbers** arranged in grids.

Why is this important?

- The convolution operation is fundamentally mathematical (multiplication and addition)
- To understand convolution, you must understand that an image is just a matrix of numbers
- This explains why CNNs process images the way they do

Real-life use:

- Every image editing app (Photoshop, Instagram filters) manipulates these numbers
 - Medical imaging (X-rays, CT scans) stores images as pixel intensity values
 - Machine learning models process these numerical arrays
-

2. Detailed Explanation

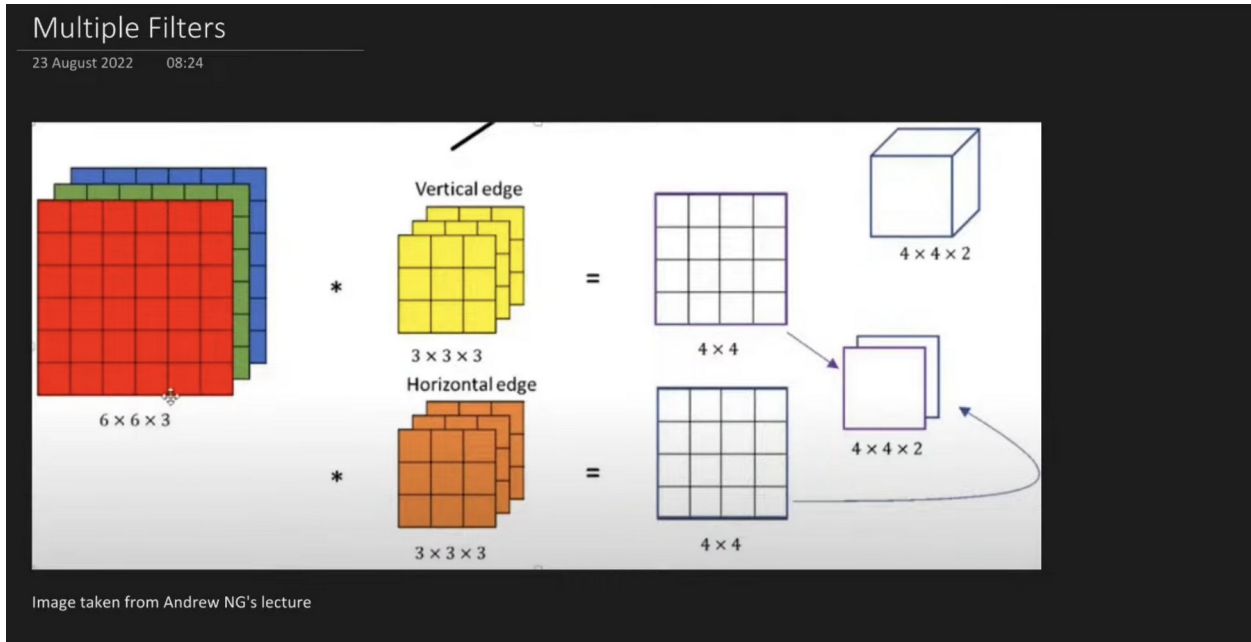
What is a Pixel?

A **pixel** is the smallest unit of a digital image. Think of it as a tiny square that contains color/brightness information.

An image is essentially a **grid** (matrix) of pixels. For example:

- A 28×28 image has **784 pixels** arranged in 28 rows and 28 columns
- A 1920×1080 image has over **2 million pixels**

Grayscale Images (Black & White)



How they are stored:

- **1 channel** (only one layer of numbers)
- Each pixel stores **one number** representing brightness

Value range: 0 to 255

- **0** = Black (complete darkness)
- **255** = White (maximum brightness)
- **Values in between** = Shades of gray

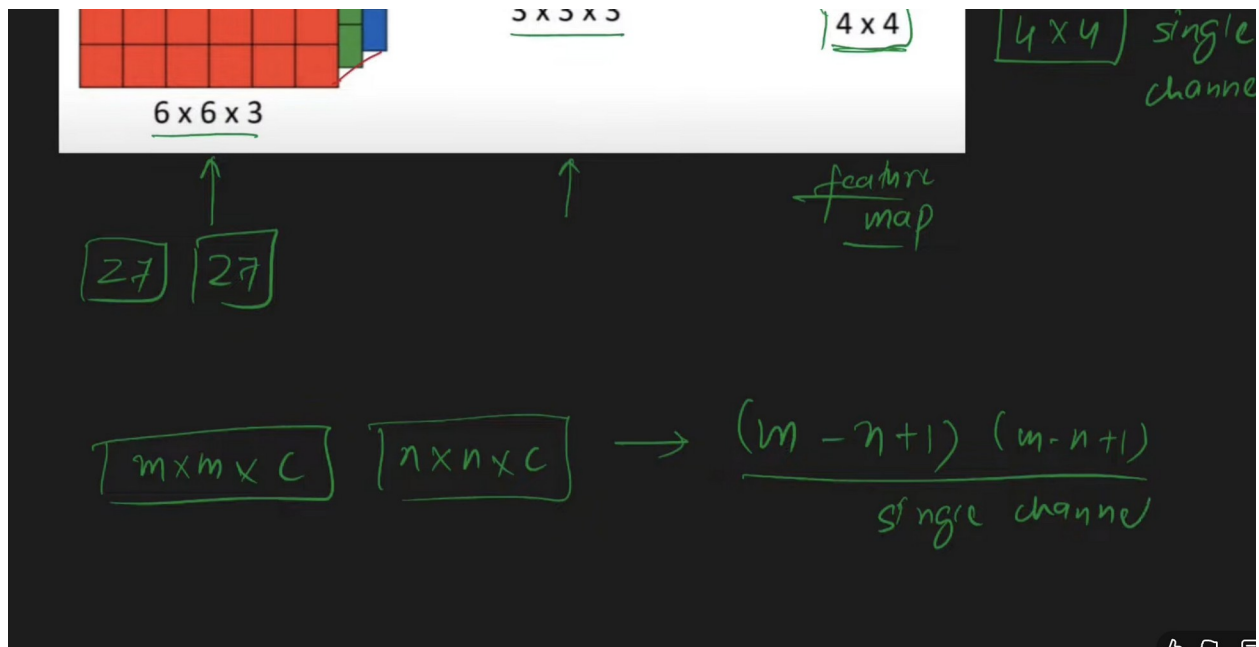
Value	Meaning
0	Pure black
128	Middle gray
255	Pure white

Example – 6×6 grayscale image:

```
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[255, 255, 255, 255, 255, 255]
[255, 255, 255, 255, 255, 255]
[255, 255, 255, 255, 255, 255]
```

This would appear as: top half black, bottom half white → a **horizontal edge**!

RGB Images (Color Images)



How they are stored:

- **3 channels** (three layers of numbers)
- **R** = Red channel, **G** = Green channel, **B** = Blue channel
- These are the **primary colors** – by combining them, we can create any color

Value range: Each channel also uses 0 to 255

- (255, 0, 0) = Pure red
- (0, 255, 0) = Pure green
- (0, 0, 255) = Pure blue
- (255, 255, 255) = White
- (0, 0, 0) = Black

Shape representation:

- Grayscale image shape: (height, width) or (height, width, 1)
- RGB image shape: (height, width, 3)

Example – 28×28 RGB image:

Shape: (28, 28, 3)

- 28 rows
- 28 columns
- 3 channels (R, G, B)

Key Difference Summary

Feature	Grayscale	RGB (Color)
Number of channels	1	3
Values per pixel	1 number	3 numbers (R,G,B)
Example value	128 (gray)	(128, 0, 128) = purple
Shape	(H, W)	(H, W, 3)

3. Key Points

- Images are stored as **matrices/arrays of numbers** in computer memory
- **Grayscale images** have 1 channel with values 0-255
- **RGB images** have 3 channels (Red, Green, Blue) with values 0-255 each
- Higher value = brighter (for grayscale) or more of that color (for RGB)
- Processing images = processing **numbers**, not actual "pictures"
- When a computer "sees" an image, it sees a multidimensional array of numbers

4. Revision Notes

Concept	Key Takeaway
Pixel	Smallest unit of an image
Grayscale	1 channel, 0-255 (0=black, 255=white)
RGB	3 channels (R,G,B), each 0-255
Image as data	A grid/matrix of numbers
Computer view	Sees numbers, not images
Shape notation	(Height, Width, Channels)

That covers image representation in computers.

Topic 3: The Convolution Operation – Core Concept

1. Introduction

What is the convolution operation? Convolution is a mathematical operation where we slide a small matrix (called a **filter** or **kernel**) over an image matrix and perform element-wise multiplication followed by summation. This produces a new matrix called a **feature map**.

Why is it important?

- This is the **most fundamental building block** of CNNs
- Its purpose is to **extract features** (edges, corners, textures) from images
- If you understand convolution, you understand the core of CNNs

Real-life use:

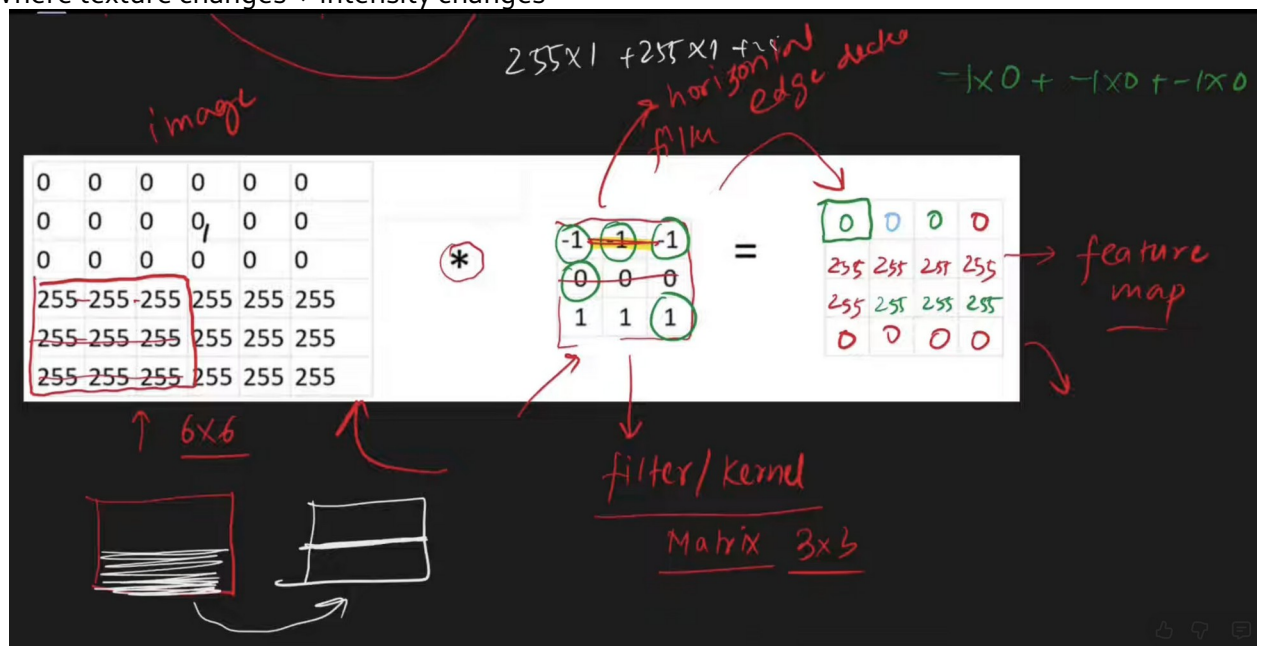
- Edge detection in photography apps
- Feature extraction in facial recognition systems
- Every CNN uses this operation to learn patterns from images

2. Detailed Explanation

What is a Feature/Edge?

An **edge** or **feature** in an image is simply a place where **intensity changes**:

- Where black meets white → intensity changes sharply
- Where colors change → intensity changes
- Where texture changes → intensity changes



Example: In a face image, edges define the boundaries of eyes, nose, lips, and face outline.

The Building Blocks

Component	Description	Analogy
Input Image	Matrix of pixel values	The canvas
Filter/Kernel	Small matrix (e.g., 3x3) with	A stencil or stamp

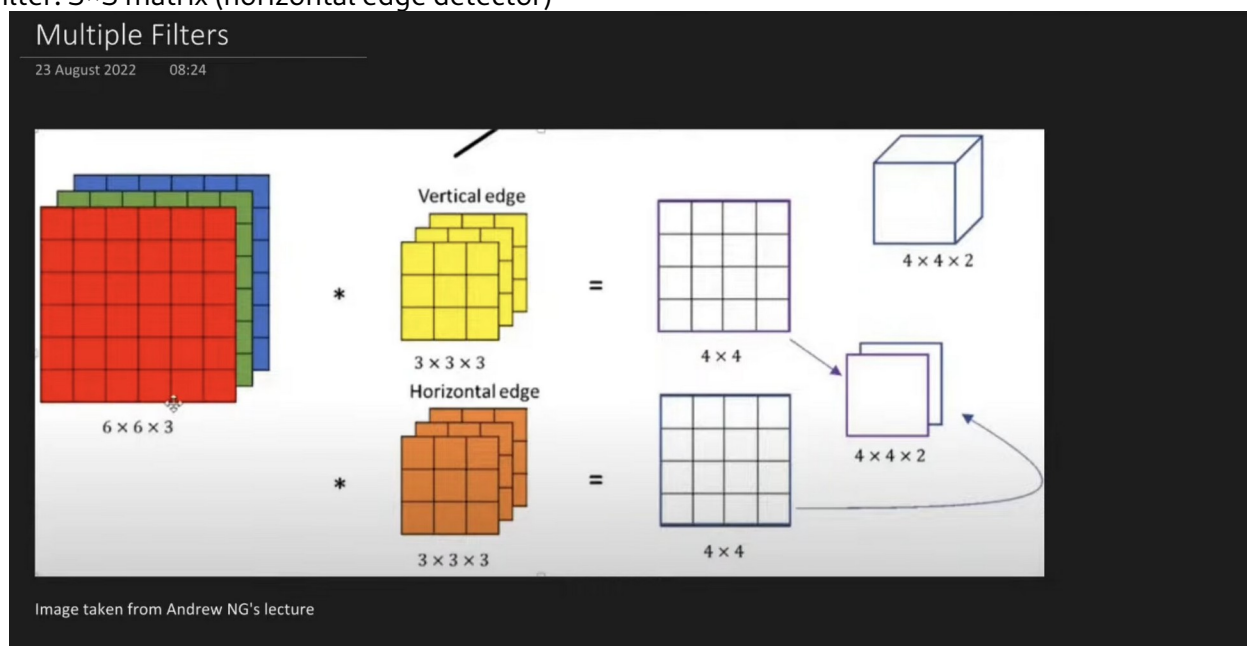
Component	Description	Analogy
	specific numbers	
Feature Map	Result after applying convolution	The traced pattern

Step-by-Step Process

Let's walk through convolution with a simple example:

Step 1: Setup

- Input Image: 6×6 matrix (grayscale)
- Filter: 3×3 matrix (horizontal edge detector)



Input Image (top half black, bottom half white):

```

Row 1: [0, 0, 0, 0, 0, 0]
Row 2: [0, 0, 0, 0, 0, 0]
Row 3: [0, 0, 0, 0, 0, 0]
Row 4: [255, 255, 255, 255, 255, 255]
Row 5: [255, 255, 255, 255, 255, 255]
Row 6: [255, 255, 255, 255, 255, 255]

```

Filter (Horizontal Edge Detector):

```

[-1, -1, -1]
[ 0,  0,  0]
[ 1,  1,  1]

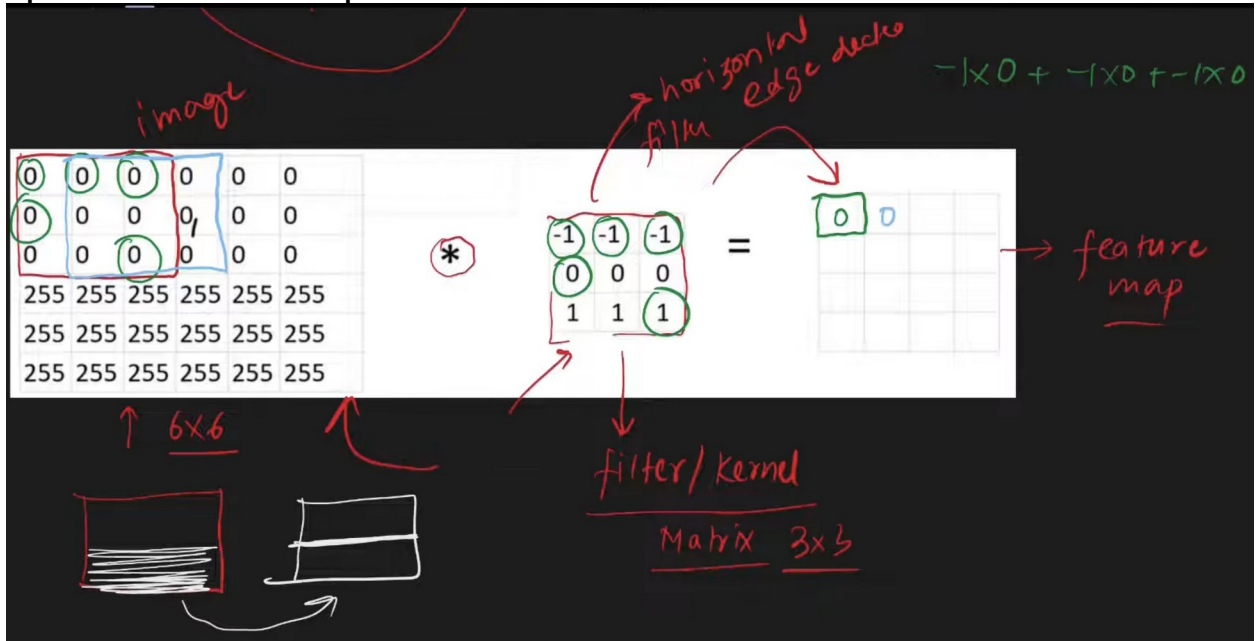
```

Step 2: Place the filter on the image (top-left corner)

The filter covers the first 3×3 region:

Image region:	Filter:
[0, 0, 0]	[-1, -1, -1]
[0, 0, 0]	[0, 0, 0]
[0, 0, 0]	[1, 1, 1]

Step 3: Element-wise multiplication



Multiply each number in the filter with the corresponding number in the image region:

$(0 \times -1) = 0$
$(0 \times -1) = 0$
$(0 \times -1) = 0$
$(0 \times 0) = 0$
$(0 \times 0) = 0$
$(0 \times 0) = 0$
$(0 \times 1) = 0$
$(0 \times 1) = 0$
$(0 \times 1) = 0$

Step 4: Sum all results

$0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 = 0$

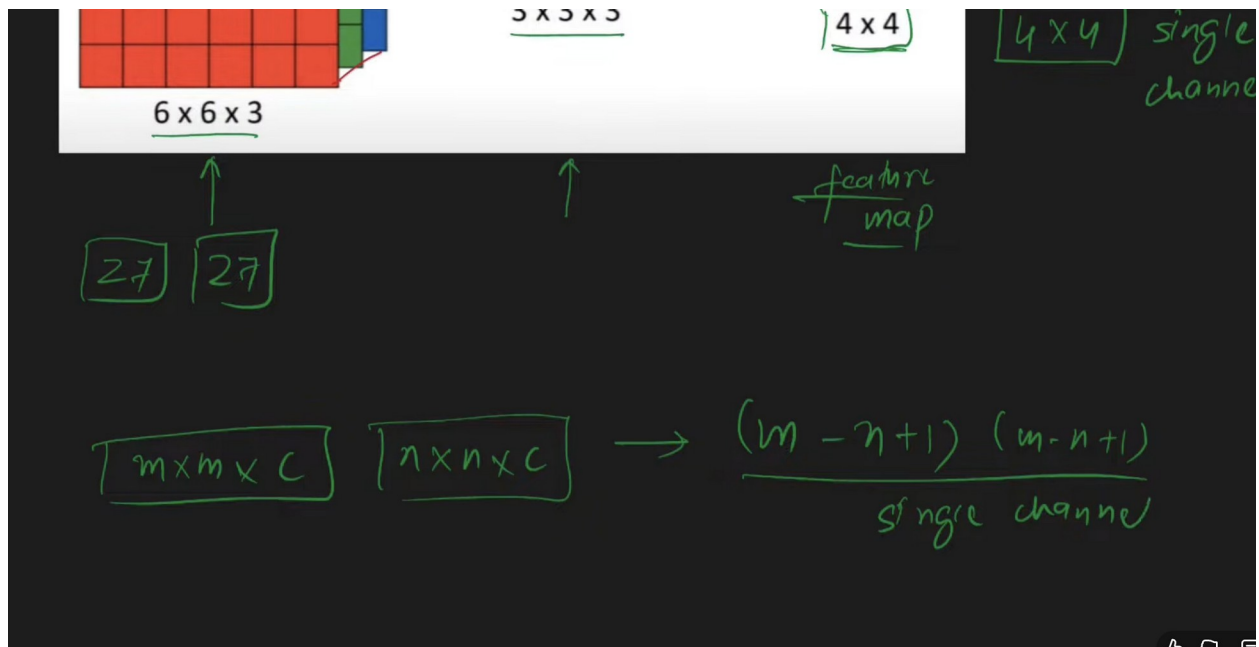
This becomes the **first value** of the feature map (top-left).

Step 5: Slide the filter to the next position

Move the filter **one step to the right** and repeat steps 2-4.

Continue sliding across the entire image, then move down and repeat.

Complete Calculation – Region Where Edge Exists



When the filter overlaps the edge (where black meets white):

Image region:	Filter:
$\begin{bmatrix} 0 & 0 & 0 \\ 255 & 255 & 255 \\ 255 & 255 & 255 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

Element-wise:

$(0 \times -1) = 0$
 $(0 \times -1) = 0$
 $(0 \times -1) = 0$
 $(255 \times 0) = 0$
 $(255 \times 0) = 0$
 $(255 \times 0) = 0$
 $(255 \times 1) = 255$
 $(255 \times 1) = 255$
 $(255 \times 1) = 255$

Sum = $0 + 0 + 0 + 0 + 0 + 0 + 255 + 255 + 255 = 765$

But wait! Values should be between 0-255. In practice, we might clip values or normalize. This demonstrates the high activation at the edge.

Output Feature Map

After sliding the 3×3 filter over the 6×6 image, we get a **4×4 feature map**:

```
[0, 0, 0, 0]
[0, 765, 765, 0]
[0, 765, 765, 0]
[0, 0, 0, 0]
```

This shows where horizontal edges are detected! The high numbers (765) indicate the location of the horizontal edge.

Visual Interpretation

- **High positive values** → Filter strongly matches the pattern (edge present)
- **Zero or low values** → No edge detected
- Different filters detect different types of features

3. Key Points

- Convolution = **element-wise multiplication + summation** while sliding a filter
- Purpose: **extract features** (edges, patterns) from images
- **Filter/Kernel** = small matrix with learned values
- **Feature Map** = output after convolution
- The filter detects where intensity changes in the image
- Positive activations indicate the presence of the feature the filter is looking for

4. Syntax/Structure

The diagram illustrates the convolution operation. It shows a 6x6 input matrix (labeled (6×6) and (28×28) with handwritten $n \times n$) multiplied by a 3x3 filter matrix (labeled (3×3) and $m \times m$) to produce a 4x4 output feature map (labeled (4×4)). Handwritten notes show the formula for the output size: $(n - m + 1) \times (n - m + 1)$.

The convolution operation formula:

For input image I (size $n \times n$) and filter F (size $f \times f$):

Output size = $(n - f + 1) \times (n - f + 1)$

Explanation:

- n = image dimension (e.g., 6)
- f = filter dimension (e.g., 3)
- Output = $6 - 3 + 1 = 4 \rightarrow 4 \times 4$ feature map

Mathematical representation for a single position:

$$\text{Output}[i, j] = \sum_a \sum_b \text{Input}[i + a, j + b] \times \text{Filter}[a, b]$$

Where:

- i, j = position in the output feature map
 - a, b = position within the filter
 - Σ = sum over all filter dimensions
-

5. Code Example

```
import numpy as np

# Simple convolution implementation
def convolve2d(image, filter):
    """
    image: 2D numpy array (grayscale image)
    filter: 2D numpy array (kernel)
    Returns: 2D numpy array (feature map)
    """
    img_height, img_width = image.shape
    filter_height, filter_width = filter.shape

    # Calculate output dimensions
    output_height = img_height - filter_height + 1
    output_width = img_width - filter_width + 1

    # Initialize output feature map
    feature_map = np.zeros((output_height, output_width))

    # Perform convolution
    for i in range(output_height):
        for j in range(output_width):
            # Extract the region of the image
            region = image[i:i+filter_height, j:j+filter_width]
            # Element-wise multiplication and sum
            feature_map[i, j] = np.sum(region * filter)

    return feature_map
```

```
# Example usage
image = np.array([
    [0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0],
    [255, 255, 255, 255, 255, 255],
    [255, 255, 255, 255, 255, 255],
    [255, 255, 255, 255, 255, 255]
])

horizontal_filter = np.array([
    [-1, -1, -1],
    [0, 0, 0],
    [1, 1, 1]
])

result = convolve2d(image, horizontal_filter)
print("Feature Map:\n", result)
```

6. Output

```
Feature Map:
[[ 0  0  0  0]
 [ 0 765 765 0]
 [ 0 765 765 0]
 [ 0  0  0  0]]
```

Explanation:

- The zeros appear where the filter saw uniform black or uniform white
- The 765 values appear exactly where the edge exists (black above, white below)
- This is how convolution highlights edges!

7. Common Mistakes

Mistake	Correction
Confusing multiplication with addition	Convolution involves both – multiply element-wise, then sum
Forgetting to slide the filter	Convolution slides the filter over every position
Thinking filters are fixed	In deep learning, filters are learned automatically
Expecting same-size output	Output size is smaller when no padding is used
Mixing up filter values	Different filter values detect different features

8. Interview/Exam Questions

Q1: What is the purpose of the convolution operation in CNNs?

To extract features (edges, textures, patterns) from input images by applying a filter and detecting where patterns match.

Q2: If we convolve a 6×6 image with a 3×3 filter, what is the size of the output feature map?

4×4. Formula: $(n - f + 1) = (6 - 3 + 1) = 4$.

Q3: Explain step-by-step how convolution is performed on a single position.

Place filter on the image region, multiply each filter value with the corresponding image pixel, then sum all products.

Q4: What do high positive values in a feature map indicate?

A strong match between the filter and that region of the image (the feature is present).

9. Revision Notes

- Convolution = **sliding window** of element-wise multiplication + summation
 - **Input → Filter → Feature Map**
 - Filter size typically 3×3, can be larger
 - Output size = $(n - f + 1) \times (n - f + 1)$
 - Purpose: detect **edges, changes in intensity, and patterns**
 - Different filters detect different features
 - In deep learning, filters are **learned automatically** during training
-

That covers the core concept of the convolution operation.

Topic 4: Filters (Kernels) and Their Role

1. Introduction

What are filters/kernels? A filter (also called a kernel) is a small matrix of numbers that we slide over an image during convolution. The values inside the filter determine **what kind of feature** the filter will detect.

Why are filters important?

- Filters are the "eyes" of the CNN – they define what patterns the network looks for
- Different filters detect different features (horizontal edges, vertical edges, corners, textures)

- In deep learning, filters are **automatically learned** during training

Real-life use:

- Image sharpening and blurring in photo editing apps
 - Edge detection in security and surveillance systems
 - Feature extraction in facial recognition
-

2. Detailed Explanation

What is a Filter?

A filter is simply a small grid of numbers. The numbers can be:

- **Positive** – detects bright patterns
- **Negative** – detects dark patterns
- **Zero** – ignores that position

Common filter sizes: 3×3 , 5×5 , 7×7

- 3×3 is most common in modern CNNs

Horizontal vs Vertical Edge Detection

Let's look at two different filters and what they detect:

Horizontal Edge Detector:

```
[ -1, -1, -1]
[  0,  0,  0]
[  1,  1,  1]
```

- **Detects:** Horizontal edges (changes from top to bottom)
- **Why:** Top row is negative, bottom row is positive – it looks for intensity change from top to bottom

Vertical Edge Detector:

```
[ -1,  0,  1]
[ -1,  0,  1]
[ -1,  0,  1]
```

- **Detects:** Vertical edges (changes from left to right)
- **Why:** Left column is negative, right column is positive – it looks for intensity change from left to right

How Different Filters Work

Filter Type	Filter Matrix	Detects
Horizontal edge	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	Horizontal lines/boundaries
Vertical edge	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	Vertical lines/boundaries
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	No change (preserves image)
Blur	$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} / 9$	Smooths image (averaging)
Sharpening	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Enhances edges

Visualizing Filter Response

When a filter is applied:

- **High positive output** → strong match (the feature exists in that region)
- **Zero or low output** → no match (the feature doesn't exist there)
- **Negative output** → opposite pattern exists (often ignored via activation functions)

Example – Horizontal filter on a face image:

- The filter strongly activates where there are horizontal edges
- Eyes, nose bridge, lips, and eyebrows all contain horizontal edges
- The feature map highlights these regions

Other Types of Filter Variations

Computer vision researchers have developed many filter types:

- **Top-edge detector**
- **Bottom-edge detector**
- **Left-edge detector**
- **Right-edge detector**
- **Diagonal-edge detectors** (top-left to bottom-right, etc.)
- **Corner detectors**

But here's the key insight:

In deep learning, we DON'T manually design these filters!

The network **learns** the optimal filter values automatically during training, based on the data it sees.

3. Key Points

- A **filter/kernel** is a small matrix of numbers that slides over the image

- Filter values determine **what feature** is detected
 - **Horizontal filters** detect horizontal edges; **vertical filters** detect vertical edges
 - Different filter values = different feature detection
 - In traditional computer vision, humans designed filters
 - **In deep learning, filters are learned automatically** – this is why CNNs are so powerful
 - The CNN learns filters that best help it solve the specific task
-

4. Code Example

```
import numpy as np

# Define different types of filters

# Horizontal edge detector
horizontal_filter = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])

# Vertical edge detector
vertical_filter = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])

# Blur filter (average of neighbors)
blur_filter = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
]) / 9

# Sharpening filter (enhances edges)
sharpen_filter = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
])

# Sobel filter (another edge detector)
sobel_horizontal = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
])
```

```
print("Horizontal filter:\n", horizontal_filter)
print("\nVertical filter:\n", vertical_filter)
print("\nBlur filter:\n", blur_filter)
print("\nSharpening filter:\n", sharpen_filter)
```

Output Explanation:

Each filter will produce a different feature map when applied to the same image:

- Horizontal filter → horizontal edges are bright
 - Vertical filter → vertical edges are bright
 - Blur filter → image becomes smoother, less detail
 - Sharpening filter → edges become more pronounced
-

5. Common Mistakes

Mistake	Correction
Thinking filter values must be designed manually	In deep learning, they are learned automatically
Assuming all filters are 3×3	Filters can be any size (3×3, 5×5, etc.)
Forgetting to sum after multiplication	Convolution = multiply then sum
Thinking only one filter is used	CNNs use hundreds/thousands of filters per layer
Confusing horizontal vs vertical filters	Horizontal: rows differ; Vertical: columns differ

6. Interview/Exam Questions

Q1: What is the difference between a horizontal edge detector and a vertical edge detector filter?

A horizontal edge detector has negative values in the top row and positive in the bottom row, detecting changes from top to bottom. A vertical edge detector has negative in the left column and positive in the right column, detecting changes from left to right.

Q2: Why is it important that CNNs learn filters automatically?

It removes the need for manual feature engineering. The network can discover the most relevant features for its specific task, often finding patterns humans wouldn't think of.

Q3: What do the values in a filter represent?

They represent weights/importance given to each position. Positive values indicate a contribution to detection, negative values suppress that contribution.

Q4: Can we use any size filter?

Yes, but 3×3 is most common because it captures local patterns effectively and is computationally efficient.

7. Revision Notes

Concept	Key Takeaway
Filter/Kernel	Small matrix of numbers for feature detection
Purpose	Detects patterns (edges, textures, shapes)
Horizontal filter	Detects horizontal edges
Vertical filter	Detects vertical edges
Learning	CNNs learn filter values automatically
Depth	Multiple filters are used per layer
Common size	3×3 is most frequent

That covers filters/kernels and their role.

Topic 5: Feature Maps, Output Size Calculation & Stride

1. Introduction

What are feature maps and output size? A **feature map** is the output produced after applying a filter to an image through convolution. It's a matrix that shows where specific features (like edges) were detected in the input. The **size** of this feature map depends on the input size, filter size, and how we slide the filter (called stride).

Why is this important?

- Understanding output size helps you design CNN architectures correctly
- You need to track dimensions through multiple layers
- It tells you how much your image "shrinks" as it passes through the network

Real-life use:

- When building a CNN, you must ensure dimensions match between layers
 - Knowing output sizes helps manage computational cost
 - Helps in debugging network architecture errors
-

2. Detailed Explanation

What is a Feature Map?

When you apply convolution to an image, you get a **feature map** (also called an **activation map**).

Key points about feature maps:

- Each feature map corresponds to **one filter**
- Each value in the feature map represents how strongly the filter matched at that position
- Higher values = stronger match (the feature exists there)
- Lower values = weak or no match

Output Size Formula

The size of the output feature map depends on three factors:

1. Input image size (n)
2. Filter size (f)
3. Stride (s) – how many pixels we move the filter each step

Basic formula (no padding):

$$\text{Output Size} = (n - f) / s + 1$$

Where:

- n = input dimension (height or width)
- f = filter dimension
- s = stride (step size)

Example Calculations

Example 1: (What we saw earlier)

- Input: 6×6 (n = 6)
- Filter: 3×3 (f = 3)
- Stride: 1 (s = 1)

$$\text{Output} = (6 - 3) / 1 + 1 = 3 + 1 = 4$$

Result: 4×4 feature map

Example 2: MNIST image

- Input: 28×28 (n = 28)
- Filter: 3×3 (f = 3)
- Stride: 1 (s = 1)

$$\text{Output} = (28 - 3) / 1 + 1 = 25 + 1 = 26$$

Result: 26×26 feature map

Example 3: With stride 2

- Input: 28×28 ($n = 28$)
- Filter: 3×3 ($f = 3$)
- Stride: 2 ($s = 2$)

$$\text{Output} = (28 - 3) / 2 + 1 = 25/2 + 1 = 12.5 + 1 = 13.5 \rightarrow 13 \text{ (floor)}$$

Note: When the division doesn't give a whole number, we take the floor. In practice, we choose settings that give clean divisions.

What is Stride?

Stride is the number of pixels the filter moves each step.

- **Stride = 1:** Filter moves 1 pixel at a time (most common)
- **Stride = 2:** Filter jumps 2 pixels at a time (reduces output size faster)

Visual analogy: Imagine stamping a pattern on a piece of paper:

- Stride 1 = Move the stamp 1 inch each time (many stamps, overlapping)
- Stride 2 = Move the stamp 2 inches each time (fewer stamps, more spaced out)

Multiple Filters – Multiple Feature Maps

Important concept: A CNN layer typically uses **multiple filters**.

- 1 filter → 1 feature map (1 channel)
- 10 filters → 10 feature maps (10 channels)
- 100 filters → 100 feature maps (100 channels)

Example:

- Input: 28×28×3 (RGB image)
- Layer uses: 10 filters (each 3×3×3)
- Output: 26×26×10 (10 feature maps)

Output Shape When Multiple Filters Are Used

If you apply **k filters** to an input:

Output shape = (Output Height) × (Output Width) × k

Where:

- Output Height = $(n - f) / s + 1$
 - Output Width = $(n - f) / s + 1$ (for square images)
 - k = number of filters used
-

3. Key Points

- **Feature map** = output of convolution
 - **Output size formula:** $(n - f) / s + 1$
 - **Stride** = how many pixels the filter jumps each step
 - **Stride 1** = overlapping windows (common)
 - **Stride > 1** = reduces output size more quickly
 - **Multiple filters** → multiple feature maps (channels in output)
 - Number of output channels = number of filters used
-

4. Syntax/Structure

Formula Summary

Variable	Meaning
n	Input size (height/width)
f	Filter size
s	Stride
k	Number of filters
Output dimension	$(n - f) / s + 1$
Output channels	k

Quick Reference Table

Input	Filter	Stride	Output
6×6	3×3	1	4×4
28×28	3×3	1	26×26
32×32	5×5	1	28×28
28×28	3×3	2	13×13
10×10	3×3	1	8×8

5. Code Example

```
import numpy as np

def calculate_output_size(n, f, s, num_filters=1):
    """
    Calculate output size of convolution

    Parameters:
    n: input dimension (assumed square)
    f: filter dimension
    s: stride
    num_filters: number of filters
```

```

Returns:
output_dim: output dimension (height/width)
output_channels: number of output channels
"""
output_dim = int((n - f) / s) + 1
return output_dim, num_filters

# Examples
examples = [
    {"n": 6, "f": 3, "s": 1, "filters": 1},
    {"n": 28, "f": 3, "s": 1, "filters": 10},
    {"n": 28, "f": 3, "s": 2, "filters": 1},
    {"n": 32, "f": 5, "s": 1, "filters": 20},
]

for ex in examples:
    dim, channels = calculate_output_size(ex["n"], ex["f"], ex["s"],
ex["filters"])
    print(f"{ex['n']}×{ex['n']} image, {ex['f']}×{ex['f']} filter,
stride {ex['s']}")
    print(f" → Output: {dim}×{dim}×{channels}\n")

```

Output:

```

6×6 image, 3×3 filter, stride 1
→ Output: 4×4×1

28×28 image, 3×3 filter, stride 1
→ Output: 26×26×10

28×28 image, 3×3 filter, stride 2
→ Output: 13×13×1

32×32 image, 5×5 filter, stride 1
→ Output: 28×28×20

```

6. Common Mistakes

Mistake	Correction
Forgetting to divide by stride	Output = (n - f)/s + 1, not n - f + 1
Using integer division incorrectly	Use floor division when the result isn't whole
Ignoring number of filters	Output channels = number of filters
Confusing feature maps with filters	Filters are used; feature maps are produced
Thinking stride only applies horizontally	Stride applies both horizontally and vertically

7. Interview/Exam Questions

Q1: What is the size of the feature map when you apply a 3×3 filter with stride 2 to a 28×28 image?

$(28 - 3)/2 + 1 = 25/2 + 1 = 12.5 + 1 = 13.5 \rightarrow 13 \times 13$ feature map.

Q2: If a convolution layer uses 64 filters on a 32×32×3 input, what is the output shape?

Output: 32×32×64 (assuming padding=1 to keep size same). Without padding:
 $(32-3)/1+1 = 30$, so 30×30×64.

Q3: What does a feature map represent?

Each feature map shows where a particular feature (pattern) was detected in the input image.

Q4: How does increasing stride affect the output?

Higher stride reduces the output size, resulting in fewer calculations and less information retained.

8. Revision Notes

- **Feature map** = output of convolution, shows feature locations
 - **Output size** = $(n - f)/s + 1$
 - **Stride** = step size (1 is common, 2+ reduces size)
 - **Multiple filters** = multiple feature maps (channels)
 - **Output channels** = number of filters used
 - Always track dimensions through the network
-

That covers feature maps, output size calculation, and stride.

Topic 6: Convolution on RGB (Color) Images

1. Introduction

What is convolution on color images? Unlike grayscale images which have only 1 channel, RGB images have **3 channels** (Red, Green, Blue). When performing convolution on RGB images, the filter also needs to have **3 channels** – one for each color channel. The operation is essentially a **3D convolution**.

Why is this important?

- Most real-world images are color images (RGB)
- Understanding multi-channel convolution is essential for working with standard image datasets
- This extends the same convolution concept to a volumetric setting

Real-life use:

- All modern computer vision systems work with RGB images
 - Self-driving cars analyze color camera feeds
 - Social media apps process color photos for filters and effects
-

2. Detailed Explanation

The 3D Nature of RGB Images

RGB Image Representation:

- Shape: (Height, Width, 3)
- Channel 0: Red intensities (0-255)
- Channel 1: Green intensities (0-255)
- Channel 2: Blue intensities (0-255)

Visualizing it: Think of an RGB image as **three grayscale images stacked together**, like layers of a cake:

- Bottom layer: Red channel
- Middle layer: Green channel
- Top layer: Blue channel

The Filter for RGB Images

Key rule: The filter must have the **same number of channels** as the input image.

- For RGB input (3 channels) → Filter must be 3 channels deep
- For grayscale input (1 channel) → Filter is 1 channel deep

Filter shape for RGB:

- Example: (3, 3, 3) → 3×3 spatial size, 3 channels deep
- In general: (f, f, input_channels)

How 3D Convolution Works

Multiple Filters

23 August 2022 08:24

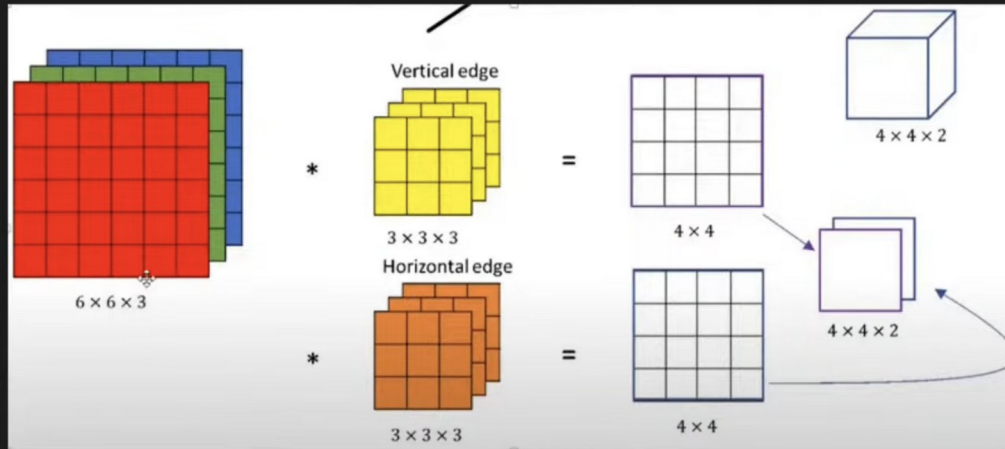
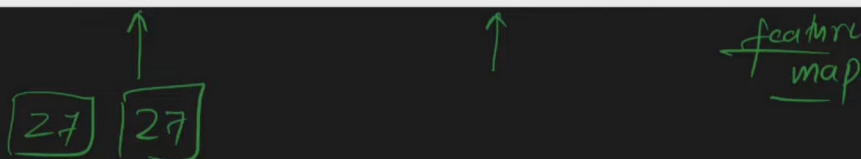


Image taken from Andrew NG's lecture



4×4 single channel



$$[m \times m \times c] [n \times n \times c] \rightarrow \frac{(m-n+1)(m-n+1)}{\text{single channel}}$$

Step-by-step process:

1. **Place the 3D filter** on the first region of the 3D image
 - The filter covers the same spatial region across ALL 3 channels
2. **Perform element-wise multiplication** for ALL positions:
 - Multiply each filter value with the corresponding image pixel in the same channel
 - This is done for Red, Green, and Blue simultaneously
3. **Sum ALL products:**
 - Add up all the multiplied results from ALL 3 channels

- This produces a **single number** for that position
4. **Slide the filter** (in 2D space) to the next position
- Move horizontally, then vertically
 - The filter moves as a 3D cube across the 2D image surface

Important insight: Even though the input has 3 channels, the output is a **single channel** feature map for each filter.

Visual Analogy

Imagine a **Rubik's Cube** (the image) and a smaller **3D block** (the filter):

- The image is a 6×6×3 cube
- The filter is a 3×3×3 block
- You place the filter on the top-left of the image
- Multiply corresponding values, sum ALL results
- Move the filter to the next position

Mathematical Representation

For RGB image I (shape: $H \times W \times 3$) and filter F (shape: $f \times f \times 3$):

$$\text{Output}[i, j] = \sum_a \sum_b \sum_c I[i+a, j+b, c] \times F[a, b, c]$$

Where:

- i, j = output position
- a, b = spatial position in filter
- c = channel index (0, 1, 2 for R, G, B)

The Magic: 3 Channels → 1 Output Channel

Why does a 3-channel input produce a single-channel output?

Because we **sum across all channels** at each position. The filter "collapses" the 3 channels into 1 number representing how well the entire 3D pattern matches.

Example:

- Input: 6×6×3
- Filter: 3×3×3
- Output: 4×4×1 (single feature map)

Output Size for RGB Convolution

The output size calculation remains the same:

$$\text{Output} = (n - f) / s + 1$$

But the **depth** (number of channels) depends on the number of filters.

3. Key Points

- RGB images have **3 channels** (Red, Green, Blue)
 - Filter for RGB must have **3 channels** (depth = input channels)
 - Convolution is performed as a **3D operation** across all channels
 - Element-wise multiplication happens across **ALL channels**, then summed
 - **3-channel input + 1 filter = 1-channel output**
 - The output is a **single feature map** per filter
 - The number of output channels = number of filters used
-

4. Syntax/Structure

Shape Summary

Component	Shape	Example
RGB Image	(H, W, 3)	(28, 28, 3)
Filter	(f, f, 3)	(3, 3, 3)
Output (1 filter)	(H-f+1, W-f+1, 1)	(26, 26, 1)
Output (k filters)	(H-f+1, W-f+1, k)	(26, 26, 10)

Rule Reminder

Always: Filter channels = Input channels

If the input changes depth, the filter depth must match.

5. Code Example

```
import numpy as np

# Simulating RGB convolution
def convolve_rgb(image, filter):
    """
    image: shape (H, W, 3) - RGB image
    filter: shape (f, f, 3) - filter with 3 channels
    Returns: feature map (single channel)
    """
    H, W, C = image.shape
    f, _, _ = filter.shape

    output_h = H - f + 1
```

```

output_w = W - f + 1

feature_map = np.zeros((output_h, output_w))

for i in range(output_h):
    for j in range(output_w):
        # Extract 3D region from image
        region = image[i:i+f, j:j+f, :]
        # Element-wise multiplication across ALL channels, then
sum
        feature_map[i, j] = np.sum(region * filter)

    return feature_map

# Create a simple 4x4 RGB image
image_rgb = np.array([
    [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[255, 0, 0], [255, 0, 0], [255, 0, 0], [255, 0, 0]],
    [[255, 0, 0], [255, 0, 0], [255, 0, 0], [255, 0, 0]]
])
# This is a red region (only Red channel has values)

# Create a 2x2x3 filter (detects red horizontal edge)
filter_rgb = np.array([
    [[-1, 0, 0], [-1, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[1, 0, 0], [1, 0, 0], [0, 0, 0]]
])

result = convolve_rgb(image_rgb, filter_rgb)
print("Feature Map:\n", result)

```

Expected Output:

```

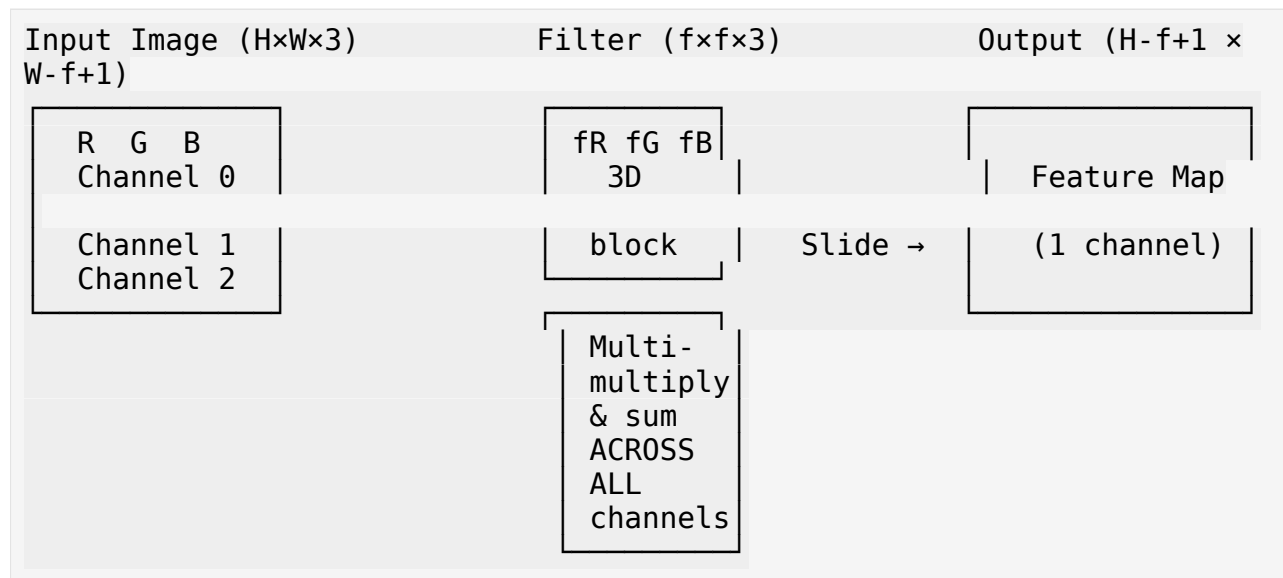
Feature Map:
[[0. 0. 0.]
 [0. 1020. 1020.]
 [0. 0. 0.]]

```

(The high numbers indicate the horizontal edge in the red channel)

6. Output

Illustration of RGB Convolution:



7. Common Mistakes

Mistake	Correction
Using a 2D filter on an RGB image	Filter must have 3 channels to match input
Thinking output also has 3 channels	Output is single-channel per filter
Separately convolving each channel	Convolution happens ACROSS all channels simultaneously
Forgetting to sum across channels	Output requires summing all channel products
Confusing filter depth with number of filters	Filter depth = input channels; Number of filters = output channels

8. Interview/Exam Questions

Q1: What happens when you apply a $3 \times 3 \times 3$ filter to a $6 \times 6 \times 3$ RGB image with stride 1?

The output is a $4 \times 4 \times 1$ feature map. The filter depth matches the input channels (3), and the spatial size reduces from 6 to 4 using the formula $(6-3)/1+1=4$.

Q2: If you apply 10 filters to an RGB image, what is the output shape?

The output will have 10 channels. For example, with $28 \times 28 \times 3$ input and $3 \times 3 \times 3$ filters, output would be $26 \times 26 \times 10$.

Q3: Why does an RGB image with 3 filters produce a single feature map?

Because we sum across all channels at each position. The 3D filter collapses the 3 channels into one number per position.

Q4: What is the shape of the filter for an input of shape (64, 64, 3)?

The filter shape would be $(f, f, 3)$ where f is the spatial size (e.g., 3, 5, 7). The third dimension must be 3.

9. Revision Notes

- RGB images = 3 channels (R, G, B)
 - Filter depth = Input depth (3 for RGB)
 - Operation is 3D: multiply across channels, then sum
 - 3-channel input + 1 filter = 1-channel feature map
 - Multiple filters = multiple feature maps (channels)
 - Shape: Input $(H, W, C) \rightarrow$ Filter $(f, f, C) \rightarrow$ Output $(H-f+1, W-f+1, \text{num_filters})$
 - Formula for output size remains the same!
-

That covers convolution on RGB (color) images.

Topic 6: Multiple Filters & Output Depth

1. Introduction

What happens when we use multiple filters? In practice, a CNN layer never uses just one filter. It uses **many filters simultaneously** (often 64, 128, 256, or more). Each filter learns to detect a **different feature** – one might detect horizontal edges, another vertical edges, another corners, and so on.

Why is this important?

- Multiple filters allow the network to capture **diverse features** from the same input
- The depth of the output (number of channels) equals the number of filters
- This creates richer representations for subsequent layers

Real-life use:

- Early CNN layers: 64 filters detect basic edges and textures
 - Middle layers: 128+ filters detect complex shapes and patterns
 - Deep layers: Hundreds of filters detect high-level concepts (eyes, wheels, etc.)
-

2. Detailed Explanation

The Concept of Multiple Filters

Single filter = Single feature:

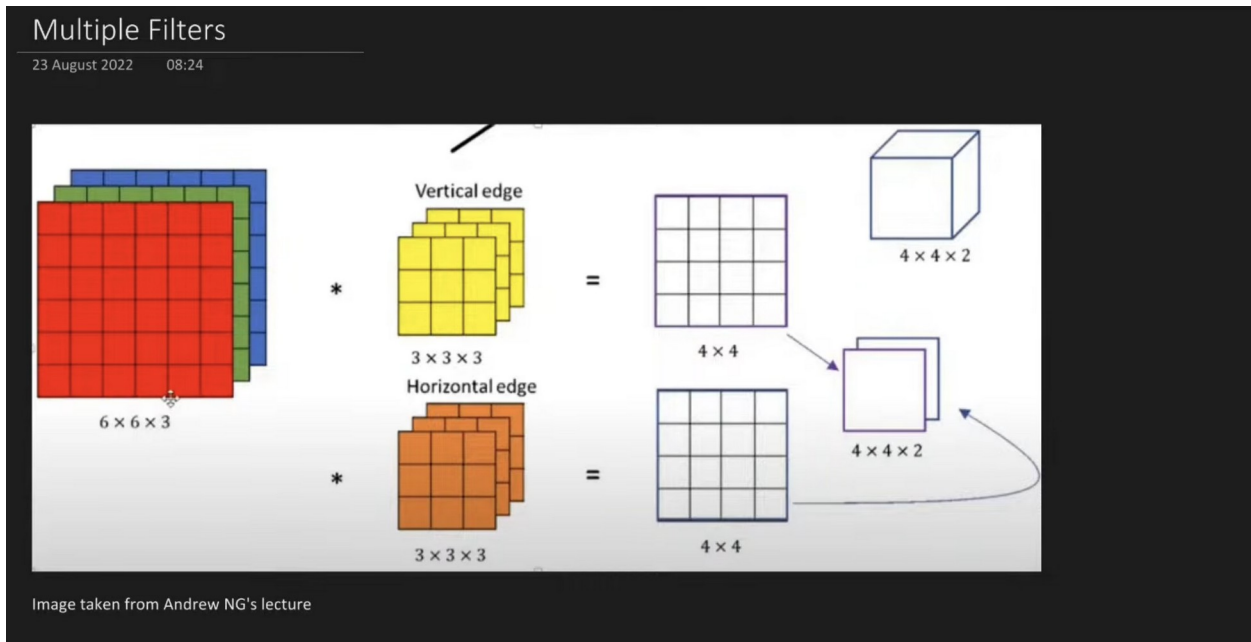
- 1 filter $\rightarrow$ detects 1 type of pattern

- Produces 1 feature map

Multiple filters = Multiple features:

- 10 filters → detects 10 different patterns
- Produces 10 feature maps (stacked together)

Example with 3 Filters



Suppose we have an RGB image and we apply **3 different filters**:

Filter	Purpose	Output
Filter 1	Horizontal edge detector	Feature Map 1
Filter 2	Vertical edge detector	Feature Map 2
Filter 3	Diagonal edge detector	Feature Map 3

Result: Instead of one feature map, we get **3 feature maps stacked together**.

Visualizing the Output

Single filter output:

Input ($H \times W \times 3$) → [Filter 1] → Feature Map 1 ($h \times w \times 1$)

Multiple filters output:

Input ($H \times W \times 3$) → [Filter 1] → Feature Map 1 ($h \times w \times 1$)
 → [Filter 2] → Feature Map 2 ($h \times w \times 1$) → Stack → Output

$(h \times w \times 3)$
→ [Filter 3] → Feature Map 3 $(h \times w \times 1)$

The Key Rule

Number of filters = Number of output channels (depth)

Formula:

Output Channels = Number of Filters Used

Shape Transformation

Example:

- Input: $28 \times 28 \times 3$ (RGB image)
- Layer: 10 filters, each $3 \times 3 \times 3$
- Output: $26 \times 26 \times 10$ (10 feature maps, stacked)

Breaking it down:

1. Filter 1: $28 \times 28 \times 3 \rightarrow 26 \times 26 \times 1$
2. Filter 2: $28 \times 28 \times 3 \rightarrow 26 \times 26 \times 1$
3. ... (repeat for all 10 filters)
4. Stack all 10 feature maps $\rightarrow 26 \times 26 \times 10$

What Do Multiple Filters Detect?

During training, each filter automatically learns to detect something different:

Layer Level	Typical Filters Learn
Early layers	Simple edges, corners, color blobs
Middle layers	Textures, patterns, simple shapes
Deep layers	Complex objects, parts of objects

Analogy: Think of filters as different colored flashlights shining on the same image:

- Red flashlight: Reveals horizontal lines
- Blue flashlight: Reveals vertical lines
- Green flashlight: Reveals corners
- Together, they give a complete picture of the image's features

Output Volume

The output of a convolution layer with multiple filters is a **volume** (3D tensor):

Output Shape = (Output Height, Output Width, Number of Filters)

This output volume becomes the **input** for the next layer.

3. Key Points

- **Multiple filters** = multiple feature maps stacked together
 - **Number of output channels** = number of filters used
 - Each filter learns a **different feature/pattern**
 - Output is a **volume** (3D tensor) with depth = number of filters
 - Using more filters → richer feature representation → more parameters
 - The network automatically learns what each filter should detect
-

4. Syntax/Structure

Shape Flow Summary

Stage	Shape	Explanation
Input	(H, W, C_in)	Original image (C_in channels)
Filter bank	(f, f, C_in, K)	K filters, each with C_in channels
Output	(H-f+1, W-f+1, K)	K feature maps stacked

Where:

- H, W = input height and width
- C_in = input channels (1 for grayscale, 3 for RGB)
- f = filter spatial size
- K = number of filters (output channels)

Important Relationship

Input Channels (C_in) → Determines filter depth
Number of Filters (K) → Determines output channels

5. Code Example

```
import numpy as np

def conv2d_multiple(image, filters):
    """
    Apply multiple filters to an image

    Parameters:
```

```

image: (H, W, C_in) - input image
filters: (K, f, f, C_in) - K filters each of size f×f×C_in

Returns:
output: (H-f+1, W-f+1, K) - K feature maps
"""
H, W, C_in = image.shape
K, f, _, _ = filters.shape

output_h = H - f + 1
output_w = W - f + 1

# Initialize output (K channels)
output = np.zeros((output_h, output_w, K))

# Apply each filter separately
for k in range(K):
    for i in range(output_h):
        for j in range(output_w):
            region = image[i:i+f, j:j+f, :]
            output[i, j, k] = np.sum(region * filters[k])

return output

# Example usage
# Input: 5×5×3 image
image = np.random.randn(5, 5, 3)

# 4 filters, each 3×3×3
filters = np.random.randn(4, 3, 3, 3)

# Apply convolution
output = conv2d_multiple(image, filters)

print(f"Input shape: {image.shape}")
print(f"Number of filters: {filters.shape[0]}")
print(f"Output shape: {output.shape}")

```

Output:

```

Input shape: (5, 5, 3)
Number of filters: 4
Output shape: (3, 3, 4)

```

Visualizing Output Depth

```

# Example with 28×28 image and 10 filters
input_shape = (28, 28, 3)
num_filters = 10

```

```

filter_size = 3

output_height = 28 - 3 + 1 # 26
output_shape = (output_height, output_height, num_filters) # (26, 26, 10)

print(f"Input: {input_shape}")
print(f"Filters: {num_filters} filters of size {filter_size}x{filter_size}x3")
print(f"Output: {output_shape}")

```

6. Common Mistakes

Mistake	Correction
Confusing number of filters with filter channels	Filter channels = input channels; Number of filters = output channels
Thinking multiple filters produce separate outputs	They are stacked into a single output volume
Forgetting that output depth equals filter count	Output channels = K (number of filters)
Assuming all filters are identical	Each filter learns different features

7. Interview/Exam Questions

Q1: If a convolution layer has 32 filters, what will be the depth of the output?

1. The output will have 32 channels, one for each filter.

Q2: What is the relationship between the number of filters and the features extracted?

Each filter learns to detect a different feature/pattern. More filters means more diverse feature detection.

Q3: A layer takes input of shape 32×32×3, uses 5×5 filters with stride 1, and has 20 filters. What is the output shape?

Output height = $(32-5)/1+1 = 28$. Output shape = 28×28×20.

Q4: Why can't we just use one filter in a CNN?

One filter would only detect one type of feature. Images contain many different patterns (edges, textures, objects), so we need many filters to capture the full complexity.

8. Revision Notes

- **Multiple filters** = multiple feature maps
 - **Output channels** = number of filters (K)
 - Each filter is **independent** and learns different features
 - Output is a **3D volume**: (H_out, W_out, K)
 - This output becomes input for the next layer
 - More filters = more parameters = more expressive power
-

That covers multiple filters and output depth.

Topic 7: Recap of the CNN Learning Process & Next Steps

1. Introduction

What is this topic about? This final section recaps the key idea of how CNNs learn, why they are powerful, and previews what comes next in the CNN learning journey. It ties together everything we've covered so far.

Why is this important?

- Understanding that CNNs learn filters automatically is the key insight that makes them so powerful
- You now have the foundation to understand the rest of CNN architecture
- It connects the theory to the practical training process

Real-life use:

- When training a CNN, you don't design filters – you let the network discover them
 - This is why CNNs outperform traditional computer vision methods
 - It's why CNNs can learn features that humans might not even think of
-

2. Detailed Explanation

The Big Idea: Automatic Filter Learning

Traditional Computer Vision:

- Humans manually design filters (like Sobel, Prewitt, etc.)
- Features are hand-crafted
- Limited to what humans can think of

Deep Learning (CNNs):

- Filters are **randomly initialized** at the start
- During training, the network **adjusts filter values** using backpropagation
- Filters become optimized for the specific task (classification, detection, etc.)
- The network discovers features that are **best for the task**

Training Process (High Level)

1. **Initialize:** Start with random filter values
2. **Forward pass:** Pass image through network (convolution operations)
3. **Calculate loss:** Compare prediction with ground truth
4. **Backward pass:** Calculate how much each filter value contributed to the error
5. **Update:** Adjust filter values to reduce error
6. **Repeat:** Go through millions of images, updating filters each time

Why This Works So Well

Analogy: Imagine you're trying to recognize faces. Instead of being told, "Look for horizontal edges here and vertical edges there," the network is shown thousands of faces and non-faces. It figures out on its own:

- "Hmm, I notice that a horizontal edge pattern with certain spacing is helpful"
- "Oh, a combination of edge detectors in a specific arrangement works better"
- "Let me adjust my filter values slightly to improve my accuracy"

Result: The CNN learns features that are:

- **Task-specific** – optimized for exactly what it needs to do
- **Hierarchical** – from simple to complex features
- **Robust** – can handle variations in lighting, rotation, scale

Summary of CNN Working

```
Step 1: Image enters the network
Step 2: Convolution layers extract features
        → Early layers: simple edges, colors
        → Deeper layers: complex patterns, objects
Step 3: Pooling layers reduce size (to be covered next)
Step 4: Fully connected layers make final decision
        → "This is a cat" or "This is a dog"
```

What We've Covered So Far

Topic	Key Takeaway
CNN Introduction	Network specialized for images
Image Representation	Images are numbers (matrices)
Convolution Operation	Slide filter, multiply, sum
Filters/Kernels	Detect features, learned automatically

Topic	Key Takeaway
Feature Maps	Output of convolution
Stride	Step size of filter sliding
RGB Convolution	3D operation across color channels
Multiple Filters	Produce multiple feature maps

What Comes Next

In upcoming videos, we'll cover:

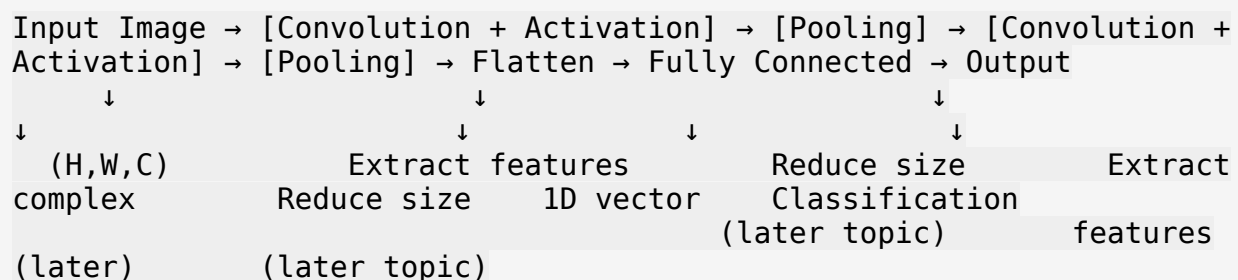
1. **Padding** – adding zeros around images to control output size
2. **Pooling** – reducing image dimensions (max pooling, average pooling)
3. **Putting it all together** – complete CNN architecture
4. **Training and optimization** – how networks learn

3. Key Points

- CNNs **learn filters automatically** – this is the key insight
- Filters start random and become optimized through training
- **Backpropagation** is used to update filter values
- The network discovers task-specific features
- This is why CNNs are more powerful than traditional methods
- CNNs build features hierarchically: edges → shapes → objects
- Multiple filters capture diverse features from the same input

4. Revision Notes (Full Summary)

CNN Architecture



Convolution Summary

Component	Purpose
Image	Input data (matrix of numbers)
Filter	Pattern detector (learned automatically)

Component	Purpose
Stride	How much filter moves each step
Feature Map	Output showing where pattern was found
Multiple Filters	Detect different patterns simultaneously

Key Formulas

What	Formula
Output Size	$(n - f) / s + 1$
Output Channels	= Number of Filters
Filter Depth	= Input Channels

5. Common Mistakes (Overall)

Mistake	Correction
Thinking filters are designed manually	In CNNs, they are LEARNED automatically
Forgetting the importance of multiple filters	One filter = one feature; many filters = rich representation
Ignoring the hierarchical nature	Features build from simple to complex
Thinking convolution is the only operation	Pooling and fully connected layers are also critical

6. Interview/Exam Questions

Q1: Why are CNNs more powerful than traditional computer vision methods?

CNNs automatically learn the best filters for the task at hand, while traditional methods rely on hand-crafted features designed by humans.

Q2: How does a CNN learn what features to detect?

Through training with backpropagation. The network adjusts filter values to minimize the error between predictions and ground truth.

Q3: What is the advantage of using multiple filters in one layer?

Each filter detects a different pattern, allowing the network to capture diverse features from the same input.

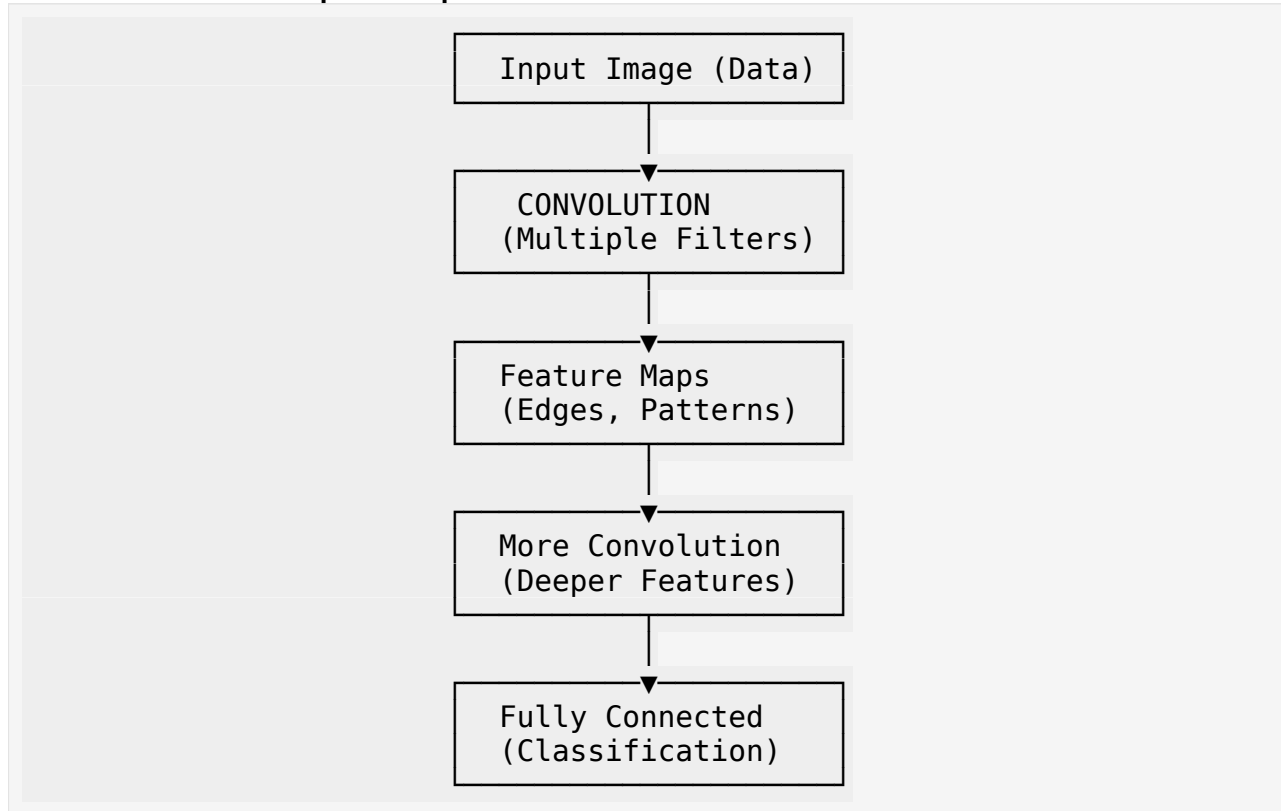
Q4: Explain the progression of features from early to deep layers.

Early layers detect simple features (edges, colors), middle layers combine these into shapes and textures, and deep layers detect complex objects and concepts.

Q5: What are the next topics to learn after convolution?

Padding, pooling layers, and complete CNN architecture.

7. Final Concept Map



That concludes all topics from the transcript!

Quick Recap of All 8 Topics:

1. Introduction to CNNs & Image Representation
2. Image Representation in Computers
3. The Convolution Operation – Core Concept
4. Filters (Kernels) and Their Role
5. Feature Maps, Output Size Calculation & Stride
6. Convolution on RGB (Color) Images
7. Multiple Filters & Output Depth
8. Recap of the CNN Learning Process & Next Steps

Next topics to look forward to: Padding, Pooling, Complete CNN Architecture, and Training Optimization.